

Lean: Machine-Checked Mathematics and Verified Programming, Past and Future

Leo de Moura
Senior Principal Applied Scientist, AWS
Chief Architect, Lean FRO

June 19, 2025

How can we ensure that our most critical software and hardware systems behave exactly as intended, and that every proof of correctness is independently verifiable?

Before Lean, there was Z3

Z3 is a state-of-the-art SMT solver.

Z3 powered **bug-finding** pipelines like Microsoft's SAGE fuzzing tool: caught thousands of defects.

But for whole-program verification, proofs were brittle: minor code edits broke solver traces.

The Lean project, started in 2013, aimed at merging interactive and automated theorem proving.



Lean is an open-source programming language and proof assistant that is transforming how we approach mathematics, software verification, and AI.

Lean opens up new possibilities for **collaboration** in mathematics.

Lean and its tooling are implemented in Lean. Lean is very **extensible**.

LSP, Parser, Macro System, Elaborator, Type Checker, Tactic Framework, Proof automation, Compiler, Build System, Documentation Authoring Tool.

Rapid self-improvement (flywheel).

Lean is based on dependent type theory from the beginning

***“Algebraic reasoning is fundamental to modern mathematics.** We calculate with **abstract structures** the same way we calculate with numbers; for example, we take sums, products, powers, and limits of structures just as we take sums, products, powers, and limits of numbers. Then, in the same breath, we talk about elements of those structures and operations on those elements. To formalize this kind of reasoning, **we need a language in which types and structures are first-class objects**, and we need tools that can interpret ambiguous notation and fill in the information that is left implicit in informal mathematics. **There is no way around using dependent type theory for all that.**”* Jeremy Avigad

Users focus on mathematics, not encoding tricks.

Lean is based on dependent type theory

An example *by Kim Morrison*:

```
structure IndexMap (α : Type u) (β : Type v) [BEq α] [Hashable α] where
  private indices : HashMap α Nat
  private keys : Array α
  private values : Array β
  private size_keys' : keys.size = values.size := by grind
  private WF : ∀ (i : Nat) (a : α), keys[i]? = some a ↔ indices[a]? = some i := by grind
```

Full example [here](#).

An example *by Kim Morrison*:

```
structure IndexMap (α : Type u) (β : Type v) [BEq α] [Hashable α] where
  private indices : HashMap α Nat
  private keys : Array α
  private values : Array β
  private size_keys' : keys.size = values.size := by grind
  private WF : ∀ (i : Nat) (a : α), keys[i]? = some a ↔ indices[a]? = some i := by grind
```

```
def insert [LawfulBEq α] (m : IndexMap α β) (a : α) (b : β) : IndexMap α β :=
  match h : m.indices[a]? with
  | some i =>
    { indices := m.indices
      keys := m.keys.set i a
      values := m.values.set i b }
  | none =>
    { indices := m.indices.insert a m.size
      keys := m.keys.push a
      values := m.values.push b }
```

An example *by Kim Morrison*:

```
/-! ### Verification theorems -/

attribute [local grind] getIdx findIdx insert

@[grind] theorem getIdx_findIdx (m : IndexMap  $\alpha$   $\beta$ ) (a :  $\alpha$ ) (h : a  $\in$  m) :
  m.getIdx (m.findIdx a h) = m[a] := by grind

@[grind] theorem mem_insert (m : IndexMap  $\alpha$   $\beta$ ) (a a' :  $\alpha$ ) (b :  $\beta$ ) :
  a'  $\in$  m.insert a b  $\Leftrightarrow$  a' = a  $\vee$  a'  $\in$  m := by
  grind

@[grind] theorem getElem_insert (m : IndexMap  $\alpha$   $\beta$ ) (a a' :  $\alpha$ ) (b :  $\beta$ ) (h : a'  $\in$  m.insert a b) :
  (m.insert a b)[a']'h = if h' : a' == a then b else m[a'] := by
  grind

@[grind] theorem findIdx_insert_self (m : IndexMap  $\alpha$   $\beta$ ) (a :  $\alpha$ ) (b :  $\beta$ ) :
  (m.insert a b).findIdx a (by grind) = if h : a  $\in$  m then m.findIdx a h else m.size := by
  grind
```




We Listen to Our Users: Classical Mathematics from Day 1

User-driven design philosophy: Classical logic and mathematics as defaults

Our first user was a mathematician: Jeremy Avigad

The math community using Lean is growing rapidly. They love the system

Lean is also a programming language, you can be constructive when it matters.

Practical focus: Verification engineers prioritize getting proofs done over foundational concerns



Mathlib

The Lean Mathematical Library supports a wide range of projects.

It is an open-source **collaborative project** with over 500 contributors and 1.8M LoC.

"I'm investing time now so that somebody in the future can have that amazing experience",

Heather Macbeth



Quanta magazine

[Physics](#)

[Mathematics](#)

[Biology](#)

[Computer Science](#)

[Topics](#)

[Archive](#)

FOUNDATIONS OF MATHEMATICS

Building the Mathematical Library of the Future



Lean is an Development Environment for formal verification

Rich user interface and integrated tooling

Build system, LSP server, and VS Code plugin work seamlessly together

Lake, Lean make, is our cargo

Reservoir (reservoir.lean-lang.org): Our package ecosystem, think crates.io

Real-time feedback: Errors, goals, and hints as you type

"Great tooling is essential" Jared Roesch

Theorem proving in Lean is an interactive game

The screenshot displays the Lean IDE interface. The left pane shows the source code for a theorem proof:

```
Odd.lean > square_of_odd_is_odd
1  import Mathlib
2
3  def odd (n : ℕ) := ∃ k, n = 2 * k + 1
4
5  -- Prove that the square of an odd number is always odd
6  theorem square_of_odd_is_odd : odd n → odd (n * n) := by
7    done
```

The right pane shows the 'Lean Infoview' with the current goal state:

```
▼ Odd.lean:7:2
▼ Tactic state
1 goal
  n : ℕ
  ⊢ odd n → odd (n * n)
```

A blue box labeled "The 'game board'" points to the goal state in the right pane.

"You have written my favorite computer game", Kevin Buzzard

Theorem proving in Lean is an interactive game

Odd.lean > `square_of_odd_is_odd`

```
1 import Mathlib
2
3 def odd (n : ℕ) := ∃ k, n = 2 * k + 1
4
5 -- Prove that the square of an odd number is always odd
6 theorem square_of_odd_is_odd : odd n → odd (n * n) := by
7   intro ⟨k1, e1⟩
8   done
9
10
11
12
```

Lean Infoview

▼ Odd.lean:8:2

▼ Tactic state

1 goal

$n \ k_1 : \mathbb{N}$
 $e_1 : n = 2 * k_1 + 1$
⊢ odd (n * n)

► Messages (1)

► All Messages (2)

A "game move", aka "tactic"

Theorem proving in Lean is an interactive game

The screenshot displays the Lean IDE interface. On the left, the source code for a theorem proof is shown:

```
Odd.lean > square_of_odd_is_odd
1  import Mathlib
2
3  def odd (n : ℕ) := ∃ k, n = 2 * k + 1
4
5  -- Prove that the square of an odd number is always odd
6  theorem square_of_odd_is_odd : odd n → odd (n * n) := by
7    intro (k1, e1)
8    simp [e1, odd]
9    done
```

On the right, the 'Lean Infoview' pane shows the current state of the proof:

- ▼ Odd.lean:9:1
- ▼ Tactic state
- 1 goal
- n k₁ : ℕ
- e₁ : n = 2 * k₁ + 1
- ├ ∃ k, (2 * k₁ + 1) * (2 * k₁ + 1) = 2 * k + 1
- Messages (1)
- All Messages (2)

The “game move” `simp`, the simplifier, is one of the most popular moves in our game

Theorem proving in Lean is an interactive game

The screenshot displays the Lean IDE interface. On the left, the editor shows a Lean script for proving that the square of an odd number is odd. The script includes an import, a definition of 'odd', a theorem statement, and a proof using the 'by' tactic. The 'use' tactic is highlighted on line 9, with a blue arrow pointing to a callout box. On the right, the 'Lean Infoview' panel shows the current goal and tactic state. The goal is to prove the equation $(2 * k_1 + 1) * (2 * k_1 + 1) = 2 * (2 * k_1 * k_1 + 2 * k_1) + 1$. The tactic state shows the current goal and the tactics used so far.

```
Odd.lean > square_of_odd_is_odd
1  import Mathlib
2
3  def odd (n : ℕ) := ∃ k, n = 2 * k + 1
4
5  -- Prove that the square of an odd number is always odd
6  theorem square_of_odd_is_odd : odd n → odd (n * n) := by
7    intro (k₁, e₁)
8    simp [e₁, odd]
9    use 2 * k₁ * k₁ + 2 * k₁
10   done
11
12
```

Lean Infoview

Odd.lean:10:1

Tactic state

1 goal

case h

$n \ k_1 : \mathbb{N}$

$e_1 : n = 2 * k_1 + 1$

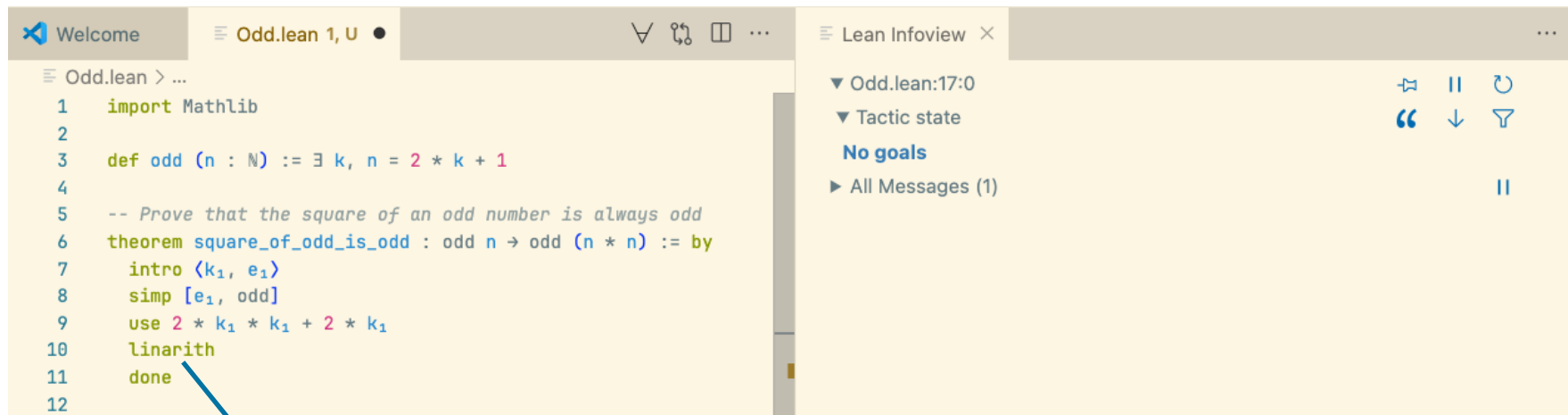
$\vdash (2 * k_1 + 1) * (2 * k_1 + 1) = 2 * (2 * k_1 * k_1 + 2 * k_1) + 1$

Messages (1)

All Messages (2)

The “game move” `use` is the standard way of proving statements about existentials

Theorem proving in Lean is an interactive game



The screenshot shows the Lean IDE interface. The main editor displays a Lean script for proving that the square of an odd number is odd. The script is as follows:

```
1 import Mathlib
2
3 def odd (n : ℕ) := ∃ k, n = 2 * k + 1
4
5 -- Prove that the square of an odd number is always odd
6 theorem square_of_odd_is_odd : odd n → odd (n * n) := by
7   intro (k₁, e₁)
8   simp [e₁, odd]
9   use 2 * k₁ * k₁ + 2 * k₁
10  linarith
11 done
12
```

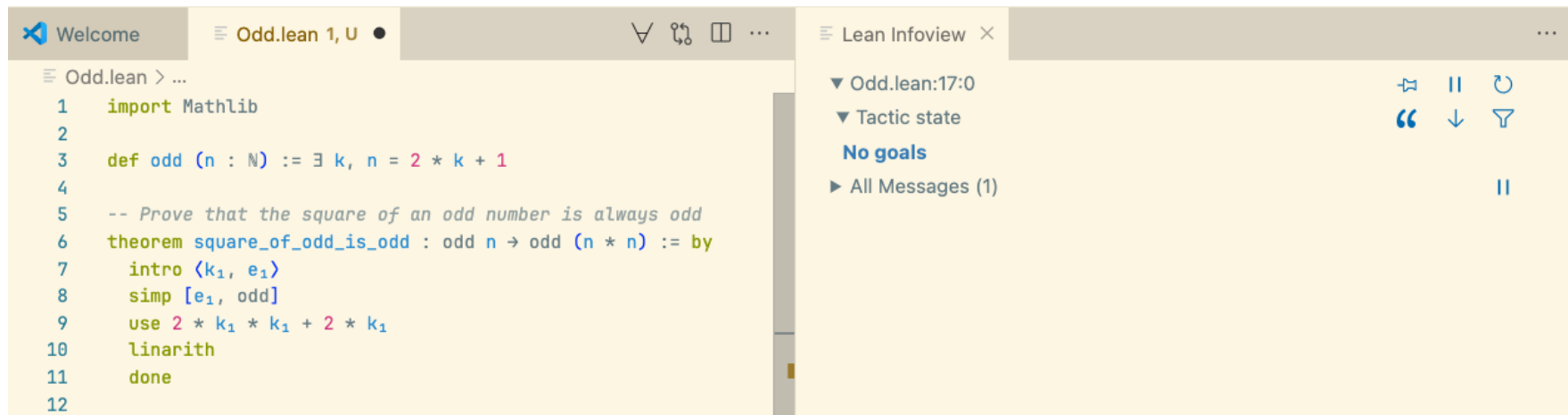
The right-hand pane shows the 'Lean Infoview' with the following content:

- Odd.lean:17:0
- Tactic state
- No goals
- All Messages (1)

Navigation icons are visible on the right side of the Infoview pane.

We complete this level using `linarith`, the linear arithmetic, move

Theorem proving in Lean is an interactive **and addictive** game



The screenshot shows the Lean IDE interface. The left pane displays the source code for a file named `Odd.lean`. The code defines an odd number and proves that its square is also odd. The right pane shows the 'Lean Infoview' for the current tactic state, indicating 'No goals' and 'All Messages (1)'. The interface includes a top bar with file tabs and a command palette, and a bottom status bar.

```
1 import Mathlib
2
3 def odd (n : ℕ) := ∃ k, n = 2 * k + 1
4
5 -- Prove that the square of an odd number is always odd
6 theorem square_of_odd_is_odd : odd n → odd (n * n) := by
7   intro ⟨k₁, e₁⟩
8   simp [e₁, odd]
9   use 2 * k₁ * k₁ + 2 * k₁
10  linarith
11  done
12
```

Lean Infoview

- ▼ Odd.lean:17:0
- ▼ Tactic state
- No goals
- All Messages (1)

"You can do 14 hours a day in it and not get tired and feel kind of high the whole day. You're constantly getting positive reinforcement", Amelia Livingston

Mathematics

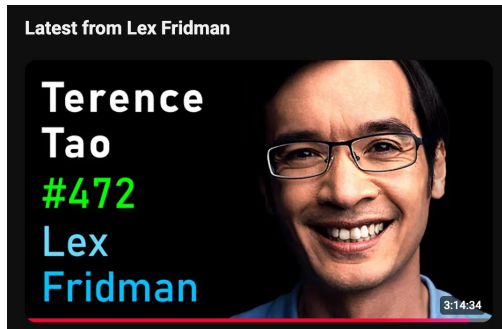
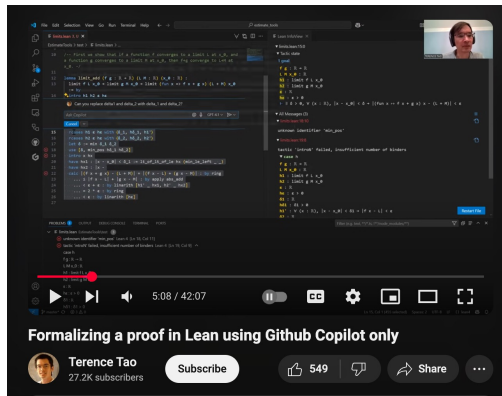
Lean is Taking Mathematics by Storm

*"Lean enables large-scale collaboration by allowing mathematicians to break down complex proofs into smaller, verifiable components. This formalization process ensures the correctness of proofs and facilitates contributions from a broader community. **With Lean, we are beginning to see how AI can accelerate the formalization of mathematics, opening up new possibilities for research.**" — Terence Tao*

Fermat's Last Theorem – Kevin Buzzard

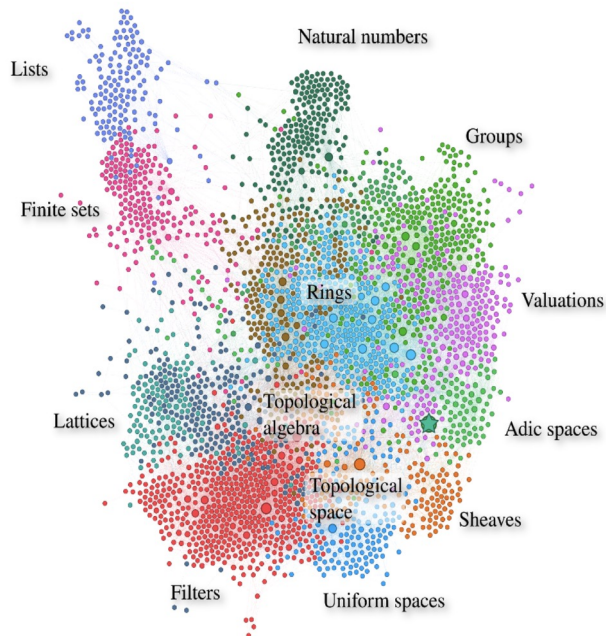
Carleson's Theorem – Floris van Doorn

How did we get here?



Preamble: the Perfectoid Spaces Project (cont.)

Kevin Buzzard, Patrick Massot, Johan Commelin



mathoverflow

Home

What are "perfectoid spaces"?



Here is a completely different kind of answer to this question.

72

A *perfectoid space* is a term of type `PerfectoidSpace` in the [Lean theorem prover](#).



Here's a quote from the source code:

```
structure perfectoid_ring (R : Type) [Huber_ring R] extends Tate_ring R : Prop :=
  (complete : is_complete_hausdorff R)
  (uniform : is_uniform R)
  (ramified : ∃ w : pseudo_uniformizer R, w^p ∉ p in R)
  (Frobenius : surjective (Frob R/p))
```



Mathlib > RingTheory > Finiteness.lean

```
355
356 theorem F6.stabilizes_of_iSup_eq {M' : Submodule R M} (hM' : M'.F6) (N : ℕ → Submodule R M)
357   (H : iSup N = M') : ∃ n, M' = N n := by
358   obtain ⟨S, hS⟩ := hM'
359   have : ∀ s : S, ∃ n, (s : M) ∈ N n := fun s =>
360     (Submodule.mem_iSup_of_chain N s).mp
361     (by
362       rw [H, ← hS]
363       exact Submodule.subset_span s.2)
364   choose f hf using this
365   use S.attach.sup f
366   apply le_antisymm
367   · conv_lhs => rw [← hS]
368     rw [Submodule.span_le]
369     intro s hs
370     exact N.2 (Finset.le_sup <| S.mem_attach ⟨s, hs⟩) (hf _)
371   · rw [← H]
372     exact le_iSup _ _
```

▼ Finiteness.lean:365:2

▼ Tactic state

1 goal

▼ case intro

R : Type u_1

M : Type u_2

inst² : Semiring R

inst¹ : AddCommMonoid M

inst : Module R M

M' : Submodule R M

N : ℕ → Submodule R M

H : iSup ↑N = M'

S : Finset M

hS : span R ↑S = M'

f : { x // x ∈ S } → ℕ

hf : ∀ (s : { x // x ∈ S }), ↑s ∈ N (f s)

⊢ ∃ n, M' = N n

Mathlib > RingTheory > Finiteness.lean

```

355
356 theorem FG.stabilizes_of_iSup_eq {M' : Submodule R M} (hM' : M'.FG) (N : ℕ → Submodule R M)
357   (H : iSup N = M') : ∃ n, M' = N n := by
358   obtain ⟨S, hS⟩ := hM'
359   have : ∀ s : S, ∃ n, (s : M) ∈ N n := fun s =>
360     (Submodule.mem_iSup_of_chain N s).mp
361     (by
362       rw [H, ← hS]
363       exact Submodule.subset_span s.2)
364   choose f hf using this
365   use S.attach.sup f
366   apply le_antisymm
367   · conv_lhs => rw [← hS]
368     rw [Submodule.span_le]
369     intro s hs
370     exact N.2 (Finset.le_sup <| S.mem_attach ⟨s, hs⟩) (hf _)
371   · rw [← H]
372     exact le_iSup _ _
---
```

▼ Finiteness.lean:365:2

▼ Tactic state

1 goal

▼ case intro

R : Type u_1

M : Type u_2

inst² : Semiring R

inst¹ : AddCommMonoid M

inst : Module R M

M' : Submodule R M

N : ℕ → Submodule R M

H : iSup ↑N = M'

S : Finset M

hS : span R ↑S = M'

f : { x // x ∈ S } → ℕ

hf : ∀ (s : { x // x ∈ S }), ↑s ∈ N (f s)

⊢ ∃ n, M' = N n



Mathlib > RingTheory > Finiteness.lean

355

356 **theorem** `FG.stabilizes_of_iSup_eq` {`M' : Submodule R M`} (`hM' : M'.FG`) (`N : ℕ → Submodule R M`)

Defs.lean ~/projects/mathlib4/Mathlib/Algebra/Module/Submodule - Definitions (1)

25 **assert_not_exists** `DivisionRing`

26

27 **open** `Function`

28

29 **universe** `u' u' u v w`

30

31 **variable** {`G : Type u''`} {`S : Type u'`} {`R : Type u`} {`M : Type v`} {`u :`

32

33 */-- A submodule of a module is one which is closed under vector oper*

34 *This is a sufficient condition for the subset of vectors in the su*

35 *to themselves form a module. -/*

36 **structure** `Submodule` (`R : Type u`) (`M : Type v`) [`Semiring R`] [`AddCommM`

37 `AddSubmonoid M, SubMulAction R M : Type v`

38

structure `Submodule` (`R : Type u`) (

▼ Finiteness.lean:356:44

▼ Expected type

`R : Type u_1`

`M : Type u_2`

`inst4 : Semiring R`

`inst3 : AddCommMonoid M`

`inst2 : Module R M`

`P : Type u_3`

`inst1 : AddCommMonoid P`

`inst0 : Module R P`

`f : M →1[R] P`

`⊢ Type u_2`

► All Messages (0)



Mathlib > Algebra > Module > Submodule > Defs.lean > Submodule

```
34   This is a sufficient condition for the subset of vectors in the submodule
35   to themselves form a module. -/
36   structure Submodule (R : Type u) (M : Type v) [Semiring R] [AddCommMonoid M] [Module R M] extends
37     AddSubmonoid M, SubMulAction R M : Type v
```

Defs.lean ~/projects/mathlib4/Mathlib/Algebra/Group/Submonoid - Definitions (1)

```
84   add_decl_doc Submonoid.toSubsemigroup
85
86   /-- `SubmonoidClass S M` says `S` is a type of subsets `s ≤ M` that
87   and are closed under `(*)` -/
88   class SubmonoidClass (S : Type*) (M : outParam Type*) [MulOneClass M]
89     MulMemClass S M, OneMemClass S M : Prop
90
91   section
92
93   /-- An additive submonoid of an additive monoid `M` is a subset containing
94   closed under addition. -/
95   structure AddSubmonoid (M : Type*) [AddZeroClass M] extends AddSubsemigroup M
96     /-- An additive submonoid contains `0`. -/
97     zero_mem' : (0 : M) ∈ carrier
98
```

▼ Defs.lean:37:8

▼ Expected type

```
G : Type u''
S : Type u'
R† : Type u
M† : Type v
ι : Type w
R : Type u
M : Type v
inst†² : Semiring R
inst†¹ : AddCommMonoid M
inst† : Module R M
⊢ Type v
```

► All Messages (0)

The Challenge

In November of 2020, Peter Scholze posits the Liquid Tensor Experiment (LTE) challenge.

*"I spent much of 2019 **obsessed** with the proof of this theorem, **almost getting crazy over it**. In the end, we were able to get an argument pinned down on paper, but I think nobody else has dared to look at the details of this, and so I still have some small lingering doubts",*

Peter Scholze

The First Victory

Johan Commelin led a team with several members of the **Lean community** and announced the **formalization of the crucial intermediate lemma** that Scholze was unsure about, with only minor corrections, in **May 2021**.

"[T]his was precisely the kind of oversight I was worried about when I asked for the formal verification. [...] The proof walks a fine line, so if some argument needs constants that are quite a bit different from what I claimed, it might have collapsed", Peter Scholze

nature

[Explore content](#) [Journal information](#) [Publish with us](#) [Subscribe](#)

[nature](#) > [news](#) > [article](#)

NEWS | 18 June 2021

**Mathematicians welcome
computer-assisted proof in 'grand
unification' theory**

Achieving the Unthinkable

The full challenge was completed in July 2022.

**The team not only verified the proof but also simplified it.
Moreover, they did this without fully understanding the entire proof.**

Johan, the project lead, reported that he could only see two steps ahead. **Lean was a guide.**

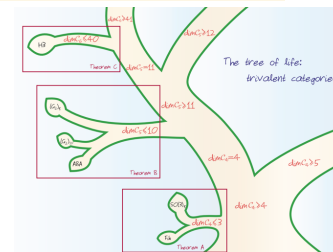
“The Lean Proof Assistant was really that: an assistant in navigating through the thick jungle that this proof is. Really, one key problem I had when I was trying to find this proof was that I was essentially unable to keep all the objects in my RAM, and I think the same problem occurs when trying to read the proof”, Peter Scholze

Automating Quantum Algebra

Here is a concrete example from quantum algebra. It comes from a classification result involving quantum $SO(3)$ categories. Specifically, the condition that certain relations among trivalent graphs imply a constraint on the parameters d , t , and c :

```
example {a} [CommRing a] [IsCharP a 0] (d t c : a) (d_inv PS03_inv : a)
  (Δ40 : d^2 * (d + t - d * t - 2) *
    (d + t + d * t) = 0)
  (Δ41 : -d^4 * (d + t - d * t - 2) *
    (2 * d + 2 * d * t - 4 * d * t^2 + 2 * d * t^4 + 2 * d^2 * t^4 - c * (d + t + d * t))) = 0)
  (_ : d * d_inv = 1)
  (_ : (d + t - d * t - 2) * PS03_inv = 1) :
  t^2 = t + 1 := by grind
```

From: “Categories generated by a trivalent vertex”, Morrison, Peters, and Snyder



Automating Quantum Algebra

```
example {α} [CommRing α] [IsCharP α 0] (d t c : α) (d_inv PS03_inv : α)
  (Δ40 : d^2 * (d + t - d * t - 2) *
    (d + t + d * t) = 0)
  (Δ41 : -d^4 * (d + t - d * t - 2) *
    (2 * d + 2 * d * t - 4 * d * t^2 + 2 * d * t^4 + 2 * d^2 * t^4 - c * (d + t + d * t))) = 0)
  (_ : d * d_inv = 1)
  (_ : (d + t - d * t - 2) * PS03_inv = 1) :
  t^2 = t + 1 := by grind
```

This is not a toy: it encodes a real algebraic constraint derived from relations among diagrams in a pivotal tensor category.

Lean can handle this kind of reasoning automatically, in [milliseconds](#).



Automating Quantum Algebra

We can explore new mathematical and physical structures, from topological quantum fields theories to fusion categories.

Lean is helping researchers reason reliably about complex symbolic systems that were previously handled only by hand or with unverified computer algebra.

grind is just another move in our interactive game.

Software

Lean in Software Verification

Lean is a programming language, and is used in **many software verification projects**.

You can write code and reason about it simultaneously.

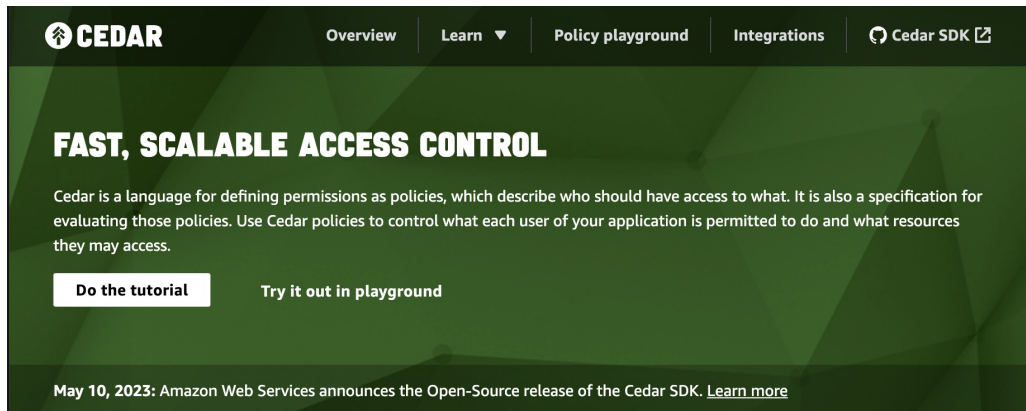
You can prove that your code has the properties you expect.

“Testing can show the presence of bugs, but not their absence”, E. Dijkstra



Cedar

<https://www.cedarpolicy.com/>

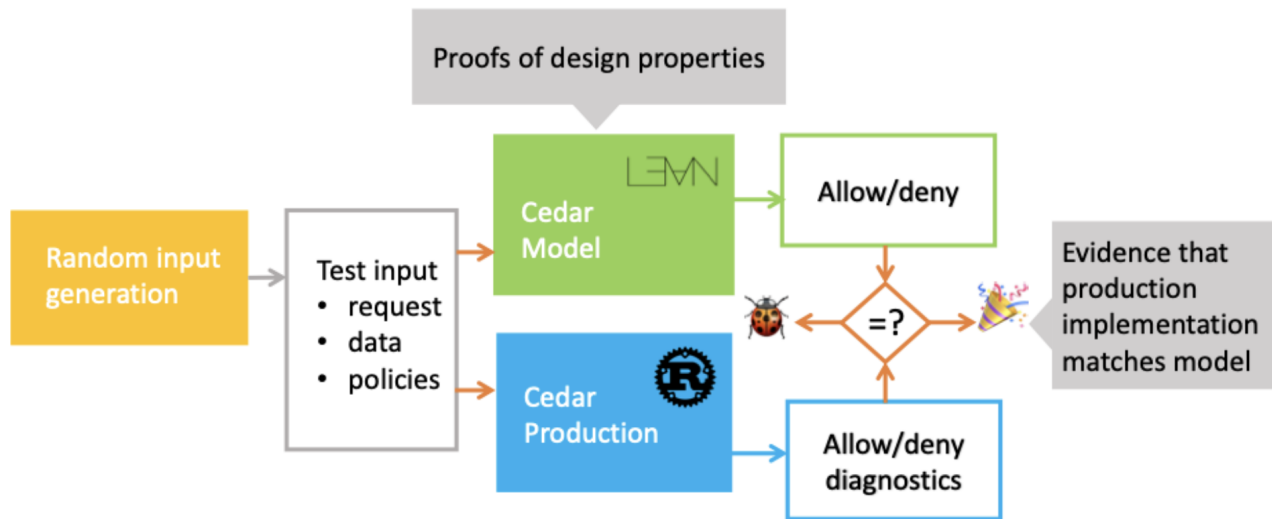


The screenshot shows the Cedar website homepage. At the top is a dark navigation bar with the Cedar logo on the left and links for Overview, Learn (with a dropdown arrow), Policy playground, Integrations, and Cedar SDK (with an external link icon). The main content area has a dark green background with a geometric pattern. It features the heading "FAST, SCALABLE ACCESS CONTROL" in white. Below this is a paragraph explaining that Cedar is a language for defining permissions as policies. Two buttons are present: "Do the tutorial" (highlighted with a white border) and "Try it out in playground". At the bottom, a dark banner contains the text "May 10, 2023: Amazon Web Services announces the Open-Source release of the Cedar SDK. [Learn more](#)".

<https://github.com/cedar-policy/cedar-spec>

```
def isAuthorized (req : Request) (entities : Entities) (policies : Policies) : Response :=
  let forbids := satisfiedPolicies .forbid policies req entities
  let permits := satisfiedPolicies .permit policies req entities
  let erroringPolicies := errorPolicies policies req entities
  if forbids.isEmpty && !permits.isEmpty
  then { decision := .allow, determiningPolicies := permits, erroringPolicies }
  else { decision := .deny, determiningPolicies := forbids, erroringPolicies }
```

Cedar



*"**Lean is the core verification technology behind Cedar**, the open-source authorization language that powers cloud services like Amazon Verified Permissions and AWS Verified Access. Our team rigorously formalizes and verifies core components of Cedar using Lean's proof assistant, and we leverage **Lean's lightning-fast runtime** to continuously test our production Rust code against the Lean formalization. Lean's efficiency, extensive libraries, and vibrant community **enable us to develop and maintain Cedar at scale**, while ensuring the key correctness and security properties that our users depend on." — Emina Torlak, Senior Principal Applied Scientist, AWS*



Cedar

To learn more about Cedar:

<https://aws.amazon.com/blogs/opensource/lean-into-verified-software-development/>

About AWSContact UsSupport▼My Account▼Sign InCreate an AWS Account

ProductsSolutionsPricingDocumentationLearnPartner NetworkAWS MarketplaceCustomer EnablementEventsExplore MoreQ

AWS Blog HomeBlogs▼Editions▼

[AWS Open Source Blog](#)

Lean Into Verified Software Development

by Kesha Hietala and Emina Torlak | on 08 APR 2024 | in [Amazon Verified Permissions](#), [Open Source](#), [Security](#), [Identity](#), & [Compliance](#), [Technical How-to](#) | [Permalink](#) | [Comments](#) | [Share](#)

Resources

[Open Source at AWS](#)
[Projects on GitHub](#)



SampCert

An **open-source** Lean library of formally **verified differential privacy primitives**.

The implementation is not only verified, but it is also **twice as fast as the previous one**.

Paper at this PLDI.

SampCert would not exist without Mathlib

SampCert is software, but its verification relies heavily on Mathlib.

The verification of code addressing practical problems in data privacy depends on the formalization of mathematical concepts, from **Fourier analysis** to **number theory** and **topology**.

“For SampCert, I started using Lean because of Mathlib, but I realized that Lean isn't just an excellent proof assistant, it's also a very pleasant and efficient programming language with a great ecosystem. As a result, we continued using Lean for TenCert.” Jean-Baptiste Tristan

Verifying Cryptography with Aeneas at Microsoft

They verify (and fix/improve) the Rust code as written by software engineers.

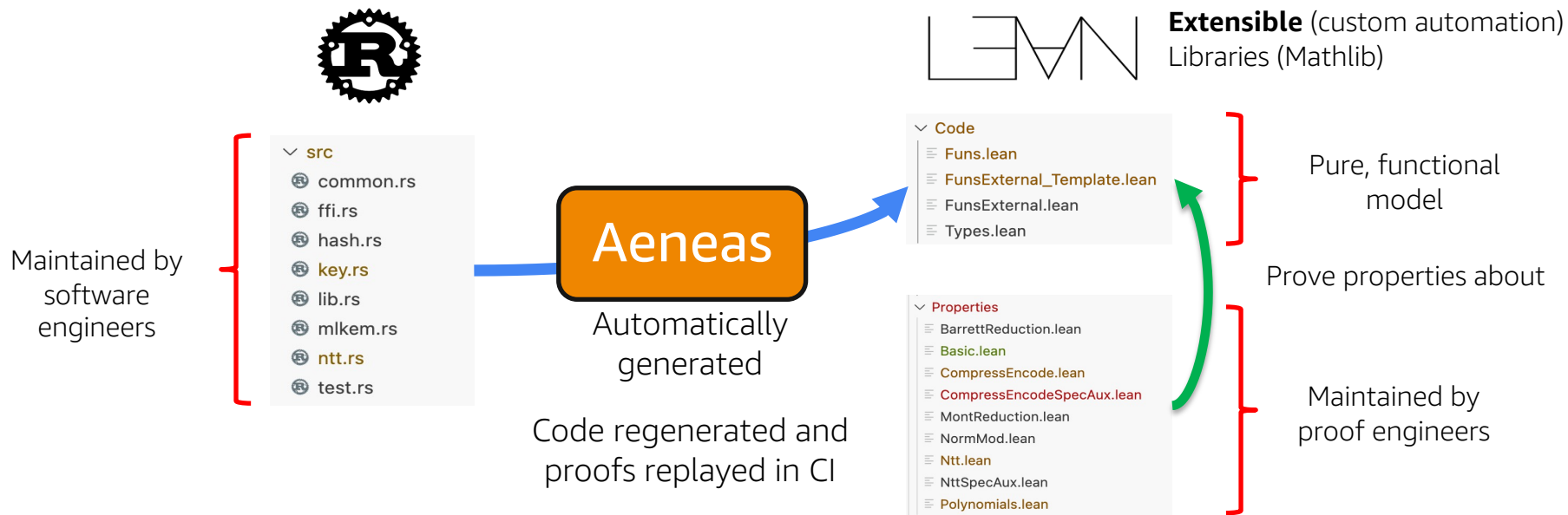
Code is evolving (new optimizations for specific hardware): They must adapt to rewrites.

[Rewriting SymCrypt in Rust to modernize Microsoft's cryptographic library.](#)

*“**The verification crucially relies on the Lean** interactive theorem prover, whose **extensibility** has been **key** in developing custom automation to make verification amenable in an industrial setting.*

Lean FRO” – Son Ho

Verifying Cryptography with Aeneas at Microsoft



Lean Extensions in Aeneas

```
syntax (name := zmodify) "zmodify" ("to" term)? ("[" (term<|>"*"),* "]" )? (location)? : tactic

def parseZModify : TSyntax ``zmodify -> TacticM (Option Expr × ScalarTac.CondSimpPartialArgs × Utils.Location)
| `(tactic| zmodify $[to $n]? $[[$args,*]]?) => do
  let n ← Utils.optElabTerm n
  let args := args.map (·.getElems) |>.getD #[]
  let args ← ScalarTac.condSimpParseArgs "zmodify" args
  pure (n, args, Utils.Location.targets #[] true)
| `(tactic| zmodify $[to $n]? $[[$args,*]]? $[$loc:location]?) => do
  let n ← Utils.optElabTerm n
  let args := args.map (·.getElems) |>.getD #[]
  let args ← ScalarTac.condSimpParseArgs "zmodify" args
  let loc ← Utils.parseOptLocation loc
  pure (n, args, loc)
| _ => Lean.Elab.throwUnsupportedSyntax
```

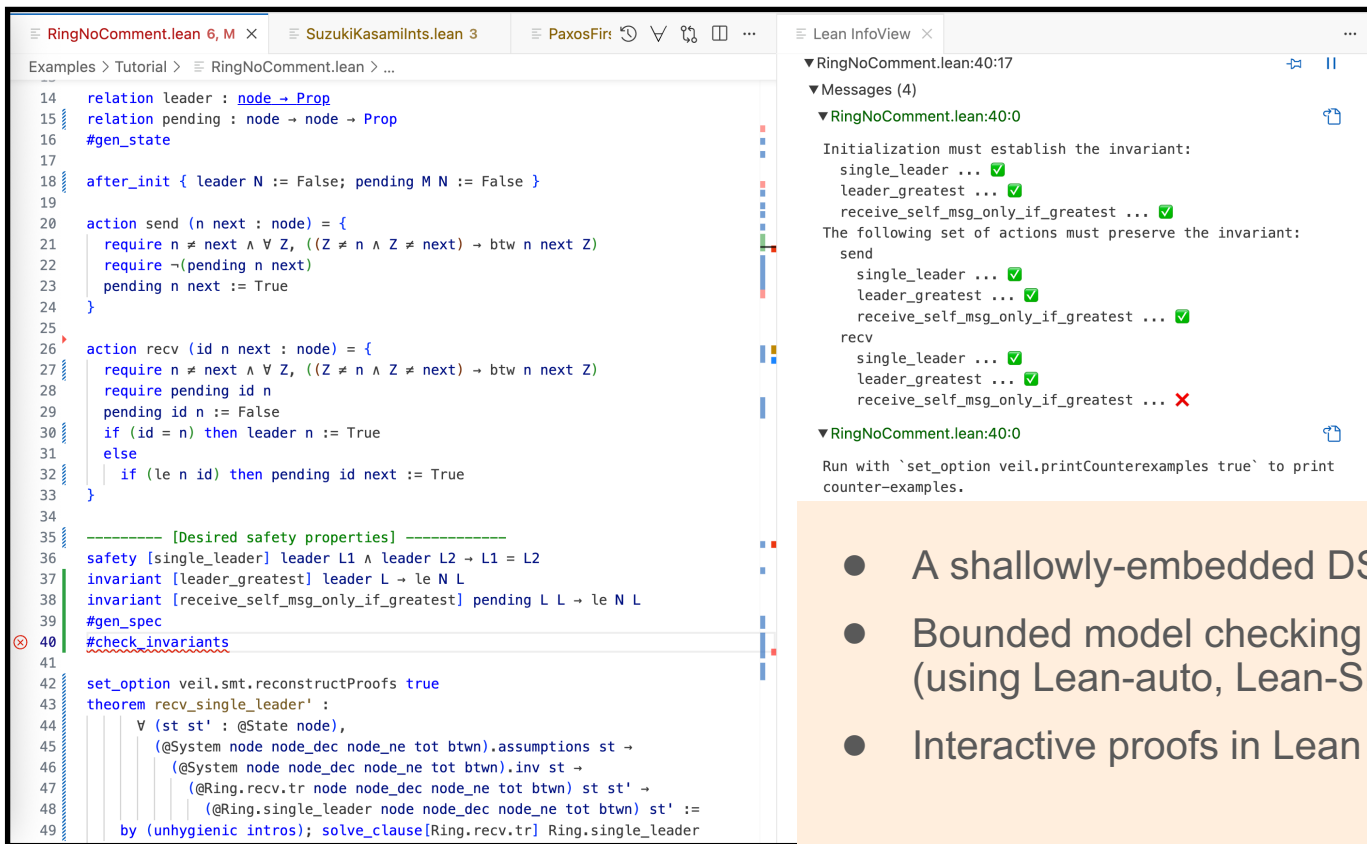
You don't need to learn a new programming language to extend Lean

Lean Extensions in Aeneas

```
-- The `scalar_tac_simps` simp attribute. -/
initialize scalarTacSimpExt : SimpExtension ←
  registerSimpAttr `scalar_tac_simps "\
    The `scalar_tac_simps` attribute registers simp lemmas to be used by `scalar_tac`
    during its preprocessing phase."
```

```
-- A simpproc to reduce expressions of the shape: `Fin.val (6 : Fin 7)` -/
simproc reduceFinOfNatVal (@Fin.val _ _) := fun e => do
  trace[ReduceFin] "- e: {e}\n"
  match e.consumeMData.getAppFnArgs with
  | (`Fin.val, #[finTy, value]) =>
    trace[ReduceFin] "- finTy: {finTy}\n- value: {value}\n"
    -- Small helper
    let extractOfNatValue (e : Expr) : Option Nat :=
      match e.consumeMData.getAppFnArgs with
      | (`OfNat.ofNat, #[_, value, _]) =>
        match exprToNat? value with
        | none => none
        | some value => some value
      | _ => none
```

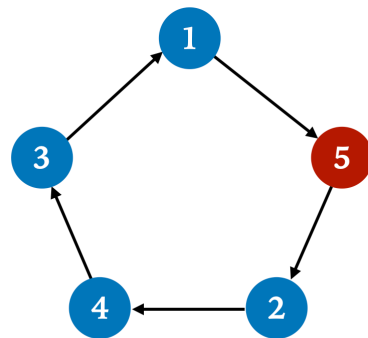
Veil: Multi-Modal Verifier for Distributed Protocols



The screenshot displays the Veil Multi-Modal Verifier interface, which consists of two main panels. The left panel shows the Lean code for a distributed protocol, with line numbers 14 through 49. The code defines a protocol with nodes, relations, and actions. The right panel shows the Lean InfoView, displaying the messages (4) and the initialization of the invariant. The messages are listed as follows:

- Initialization must establish the invariant:
 - single_leader ... ✓
 - leader_greatest ... ✓
 - receive_self_msg_only_if_greatest ... ✓
- The following set of actions must preserve the invariant:
 - send
 - single_leader ... ✓
 - leader_greatest ... ✓
 - receive_self_msg_only_if_greatest ... ✓
 - recv
 - single_leader ... ✓
 - leader_greatest ... ✓
 - receive_self_msg_only_if_greatest ... ✗

The bottom of the right panel shows the command to run the verification: `Run with 'set_option veil.printCounterexamples true' to print counter-examples.`



There is at most one leader.

- A shallowly-embedded DSL in Lean
- Bounded model checking and automation via SMT (using Lean-auto, Lean-SMT)
- Interactive proofs in Lean when automation fails

Splean: a Simple Separation Logic in Lean

The screenshot displays the Lean IDE interface with the Splean library open. The editor shows the definition of the `add_pointer` function and its specification lemma `add_pointer_spec`. The tactic state on the right shows the goal `xsimp_goal.a.a.a` with the current state of the proof.

```
8 section find_index
25 namespace Lang
53
54 lang_def add_pointer :=
55   fun p q =>
56     let m := !q in
57     for i in [0:m] {
58       ++p
59     }; !p
60
61 lemma add_pointer_spec (p q : loc) (n m : Int) ( _ : m >= 0 ) :
62   { p ~> n * q ~> m }
63   [ add_pointer p q ]
64   { v, 'v = n+m' * T } := by
65     xstep
66     -- Tactic for a [for]-loop. This tactic should be supplied with a loop
67     -- invariant [I]. In this case [I] only captures a piece of the state,
68     -- relevant to the loop body. The rest of the state would be framed
69     -- automatically
70     xfor (fun i => p ~> n + i)
71     { move=>*;
72       xapp; -- Here rather than symbolically executing the
73             -- top-most instruction in [incr], we apply its
74             -- specification lemma directly via [xapp] tactic
75       xsimp; omega }
76     move=> ? /=; xapp; xsimp
77
```

The tactic state on the right shows the goal `xsimp_goal.a.a.a` with the current state of the proof:

```
▼ Misc.lean:65:9
▼ Tactic state
1 goal
▼ case xsimp_goal.a.a.a
p q : loc
n m : ℤ
x† : m ≥ 0
├ { q ~> m * p ~> n }
  [for i in [0 : m] {
    ++p };
    !p]
  { v, 'v = (n + m)† * τ }
► All Messages (1)
```

Restart File

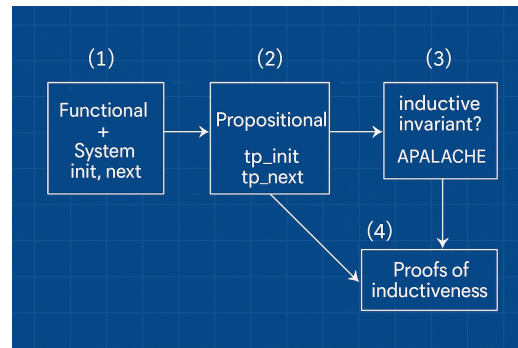
More Protocol Verification in Lean

"No other interactive theorem prover captured my attention for so long." Igor Konnov

[Specifying and simulating two-phase commit in Lean4](#)

[Proving consistency of two-phase commit in Lean4](#)

[Proving completeness of an eventually perfect failure detector in Lean4](#)



KLR: a language and elaborators for machine learning kernels

Define a common representation for kernel functions with a precise formal semantics along with translations from common kernel languages to the KLR core language.

"The lean meta programming is amazing. Have managed to delete hundreds of lines of boilerplate in the last couple days." Sean McLaughlin

KLR is also [open source](#).

```
private def evalTensorScalar (ts : TensorScalar) (t: ByteArray) : Err ByteArray := do
  match ts with
  | TensorScalar.mk op0 c0 rev0 op1 c1 rev1 =>
    let f0 <- evalAluOp op0
    let f1 <- evalAluOp op1
    let c0 := c0.toLEByteArray
    let c1 := c1.toLEByteArray
    apply2 f0 rev0 c0 f1 rev1 c1 t
```

KLR: a language and elaborators for machine learning kernels

KLR uses bit-vectors, fixed integers, etc.

```
private def decBV64 : DecodeM (BitVec 64) :=
  let u8_64 : DecodeM UInt64 := next >=> fun x => return x.toUInt64
  return ((<- u8_64) <<< 0 |||
    (<- u8_64) <<< 8 |||
    (<- u8_64) <<< 16 |||
    (<- u8_64) <<< 24 |||
    (<- u8_64) <<< 32 |||
    (<- u8_64) <<< 40 |||
    (<- u8_64) <<< 48 |||
    (<- u8_64) <<< 56).toBitVec
```

bv_decide: another powerful move

A verified bit-blaster by **Henrik Boving**, Josh Clune, Siddharth Bhat, and Alex Keizer

Uses LRAT proof producing SAT solvers: **Cadical**

```
/-  
Close a goal by:  
1. Turning it into a BitVec problem.  
2. Using bitblasting to turn that into a SAT problem.  
3. Running an external SAT solver on it and obtaining an LRAT proof from it.  
4. Verifying the LRAT proof using proof by reflection.  
-/  
syntax (name := bvDecideSyntax) "bv_decide" : tactic
```

“Blasting” popcount with bv_decide

```
def popcount : Stmt := imp {
  x := x - ((x >>> 1) &&& 0x55555555);
  x := (x &&& 0x33333333) + ((x >>> 2) &&& 0x33333333);
  x := (x + (x >>> 4)) &&& 0x0F0F0F0F;
  x := x + (x >>> 8);
  x := x + (x >>> 16);
  x := x &&& 0x0000003F;
}
```

```
def pop_spec (x : BitVec 32) : BitVec 32 :=
  go x 0 32
where
  go (x : BitVec 32) (pop : BitVec 32) (i : Nat) : BitVec 32 :=
    match i with
    | 0 => pop
    | i + 1 =>
      let pop := pop + (x &&& 1#32)
      go (x >>> 1#32) pop i
```

```
theorem popcount_correct :
```

```
  ∃ p, (run (Env.init x) popcount 8) = some p ∧ p "x" = pop_spec x := by
  simp [run, popcount, Expr.eval, Expr.BinOp.apply, Env.set, Value, pop_spec, pop_spec.go]
  bv_decide
```


“Blasting” popcount with bv_decide

Imp.lean > {} Imp.Stmt > popcount_correct

```
50 theorem popcount_correct :
51   ∃ p, (run (Env.init x) popcount 8) = some p
52   simp [run, popcount, Expr.eval, Expr.BinOp.app
53   bv_decide
54
```

▼Tactic state

1 goal

```
x : Value
├ ((x - (x >> 1 &&& 1431655765#32) &&& 858993459#32) + ((x - (x >> 1 &&&
1431655765#32)) >> 2 &&& 858993459#32) +
  ((x - (x >> 1 &&& 1431655765#32) &&& 858993459#32) +
    ((x - (x >> 1 &&& 1431655765#32)) >> 2 &&& 858993459#32)) >>
      4 &&&
        252645135#32) +
  ((x - (x >> 1 &&& 1431655765#32) &&& 858993459#32) +
    ((x - (x >> 1 &&& 1431655765#32)) >> 2 &&& 858993459#32) +
  ((x - (x >> 1 &&& 1431655765#32) &&& 858993459#32) +
    ((x - (x >> 1 &&& 1431655765#32)) >> 2 &&& 858993459#32)) >>
      4 &&&
        252645135#32) >>
      8 +
  (((x - (x >> 1 &&& 1431655765#32) &&& 858993459#32) +
    ((x - (x >> 1 &&& 1431655765#32)) >> 2 &&& 858993459#32) +
  ((x - (x >> 1 &&& 1431655765#32) &&& 858993459#32) +
    ((x - (x >> 1 &&& 1431655765#32)) >> 2 &&& 858993459#32)) >>
      4 &&&
        252645135#32) +
  ((x - (x >> 1 &&& 1431655765#32) &&& 858993459#32) +
    ((x - (x >> 1 &&& 1431655765#32)) >> 2 &&& 858993459#32) +
  ((x - (x >> 1 &&& 1431655765#32) &&& 858993459#32) +
    ((x - (x >> 1 &&& 1431655765#32)) >> 2 &&& 858993459#32)) >>
      4 &&&
        252645135#32) >>
      8) >>
    16 &&&
      63#32 =
  (x &&& 1#32) + (x >> 1 &&& 1#32) + (x >> 2 &&& 1#32) + (x >> 3 &&& 1#32) + (x >>
4 &&& 1#32) +
```

Does Lean Have Hammers?

The Lean community is also actively developing automation.

[LeanHammer](#): an automated reasoning tool for Lean which brings together multiple proof search and reconstruction techniques and combine them into one tool.

[Lean-SMT: An SMT tactic for discharging proof goals in Lean](#)

*"Improving automation for proofs in Lean is an exciting research direction. **Lean-SMT aims to improve automation by enabling the automatic replay in Lean of proof certificates produced** by SMT solvers."*

Clark Barrett

grind (again)

```
example (x : BitVec 16) : (x + 256)*(x - 256) = x^2 := by
  grind
```

```
def siftDown (a : Array Int) (root : Nat) (e : Nat) (h : e ≤ a.size := by grind) : Array Int :=
  if _ : leftChild root < e then
    let child := leftChild root
    let child := if _ : child+1 < e then
      if a[child] < a[child + 1] then child + 1 else child
    else child
    if a[root] < a[child] then
      let a := a.swap root child
      siftDown a child e
    else a
  else a
termination_by e - root

theorem siftDown_size {a root e h} : (siftDown a root e h).size = a.size := by
  fun_induction siftDown <=> grind [siftDown]
```

grind diagnostics at your fingertips

```
example {α} (as bs cs : Array α) (v₁ v₂ : α)
  (i₁ i₂ j : Nat)
  (h₁ : i₁ < as.size)
  (h₂ : bs = as.set i₁ v₁)
  (h₃ : i₂ < bs.size)
  (h₄ : cs = bs.set i₂ v₂)
  (h₅ : i₁ ≠ j)
  (h₆ : j < cs.size)
  (h₆ : j < as.size)
  : cs[j] = as[j] := by
```

grind

```
`grind` failed
▼case grind
α : Type u_1
as bs cs : Array α
v₁ v₂ : α
i₁ i₂ j : Nat
h₁ : i₁ + 1 ≤ as.size
h₂ : bs = as.set i₁ v₁ ...
h₃ : i₂ + 1 ≤ bs.size
h₄_1 : cs = bs.set i₂ v₂ ...
h₄ : ¬i₁ = j
h₅ : j + 1 ≤ cs.size
h₆ : j + 1 ≤ as.size
h : ¬cs[j] = as[j]
└ False
```

```
[grind] Goal diagnostics ▼
[facts] Asserted facts ►
[eqc] True propositions ►
[eqc] False propositions ►
[eqc] Equivalence classes ►
[ematch] E-matching patterns ►
[cutsat] Assignment satisfying linear constraints ▼
  [assign] i₁ := 0
  [assign] i₂ := 1
  [assign] j := 1
  [assign] as.size := 2
  [assign] bs.size := 2
  [assign] cs.size := 2
```



"if-normalization" challenge by Leino, Merz, and Shankar

```
def normalize (assign : Std.HashMap Nat Bool) : IfExpr → IfExpr
| lit b => lit b
| var v =>
  match assign[v]? with
  | none => var v
  | some b => lit b
| ite (lit true) t _ => normalize assign t
| ite (lit false) _ e => normalize assign e
| ite (ite a b c) t e => normalize assign (ite a (ite b t e) (ite c t e))
| ite (var v) t e =>
  match assign[v]? with
  | none =>
    let t' := normalize (assign.insert v true) t
    let e' := normalize (assign.insert v false) e
    if t' = e' then t' else ite (var v) t' e'
  | some b => normalize assign (ite (lit b) t e)
termination_by e => e.normSize

-- We tell `grind` to unfold our definitions above.
attribute [local grind] normalized hasNestedIf hasConstantIf hasRedundantIf disjoint vars eval List.disjoint

theorem normalize_spec (assign : Std.HashMap Nat Bool) (e : IfExpr) :
  (normalize assign e).normalized
  ∧ (∀ f, (normalize assign e).eval f = e.eval fun w => assign[w]?.getD (f w))
  ∧ ∀ (v : Nat), v ∈ vars (normalize assign e) → ¬ v ∈ assign := by
  fun_induction normalize with grind
```

AI



Lean Enables **Verified** AI for Mathematics and Code

LLMs are powerful tools, but they are prone to **hallucinations**.

In Math, a **small mistake can invalidate the whole proof**.

Imagine manually checking an AI-generated proof with the size and complexity of FLT.

The informal proof is **over 200 pages**.

Buzzard estimates a formal proof will require more than **1M LoC** on top of Mathlib.

Machine-checkable proofs are the antidote to hallucinations.

Synthetic Data Generation

LLMs require **vast amounts of data** for training.

Lean mathematical libraries provide valuable, **correct-by-construction training data**.

Tools like [lean-training-data](#), by **Kim Morrison**, extract data that includes the “game board” before and after each “move”.

[Pantograph](#) by Leni Aniva (Stanford) is also getting very popular in the Lean community.

AlLean, a project led by **Soonho Kong** at AWS, uses Lean to generate **new synthetic theorems** that are correct by construction.

AI Proof Assistants

Several groups are developing AI that suggests the **next move**(s) in Lean's interactive proof game.

[LeanDojo](#) is an open-source project from Caltech, and everything (model, datasets, code) is open.

[OpenAI](#) and [Meta AI](#) have also developed AI assistants for Lean.

Claude 4 is fantastic on Lean code. Their [System Card](#) contains a Lean example.



The New York Times

Artificial Intelligence >

A.I.'s Math Problem

A.I. Training Data Disappears

Microsoft's Risk-Taker

Fine Print Changes

Quiz: Fake or Real Images?

Move Over, Mathematicians, Here Comes AlphaProof

A.I. is getting good at math — and might soon make a worthy collaborator for humans.

Share full article



47



Ringling the gong at Google DeepMind's London headquarters, a ritual to celebrate each A.I. milestone, including its recent triumph of reasoning at the International Mathematical Olympiad. Google DeepMind

google-deepmind / formal-conjectures

Code Issues 48 Pull requests 21 Actions Projects Security Insights

formal-conjectures Public Watch 16 Fork 43 Star 478

main Go to file Code

Rekile and Paul-Lez Fix: AMS codes (#185) ed8a809 - yesterday

.devcontainer	feat: Add gitpod integration (#181)	2 days ago
.github	Fix caching issues with the doc build...	last week
.vscode	vscode settings (#164)	5 days ago
FormalConjectures	Fix: AMS codes (#185)	yesterday
docbuild	Fix caching issues with the doc build...	last week
scripts	ci: add a copyright header check (#...	2 weeks ago
.gitignore	move OpenProblems to third_party	2 months ago
.gitpod.yml	feat: Add gitpod integration (#181)	2 days ago
.mailmap	chore: add .mailmap (#60)	2 weeks ago

About

A collection of formalized statements of conjectures in Lean.

[google-deepmind.github.io/fo...](#)

[formal-mathematics](#) [lean4](#)

Readme

Apache-2.0 license

Activity

Custom properties

478 stars

16 watching

43 forks

Report repository

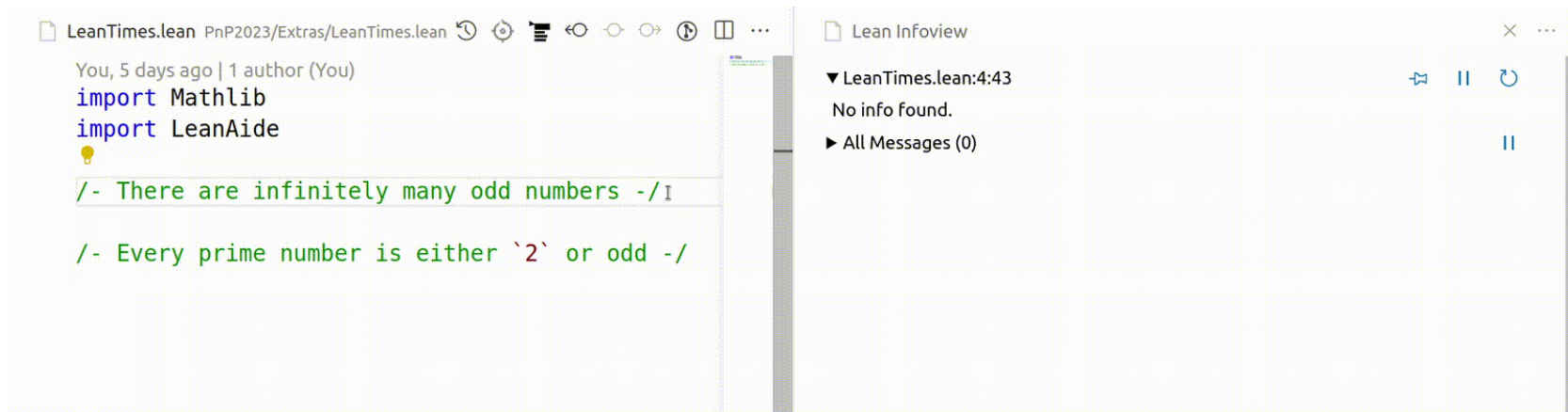
"At Google DeepMind, we used Lean to build AlphaProof, a new reinforcement-learning based system for formal math reasoning. **Lean's extensibility and verification capabilities were key in enabling the development of AlphaProof.**" — Pushmeet Kohli, Vice President, Research Google DeepMind

Auto-formalization

The process of converting natural language into a formal language like Lean.

It is much **easier to learn to read Lean than to write it**.

[LeanAide](#) is one of the auto-formalization tools available for Lean.



```
LeanTimes.lean PnP2023/Extras/LeanTimes.lean
You, 5 days ago | 1 author (You)
import Mathlib
import LeanAide
💡
/- There are infinitely many odd numbers -/
/- Every prime number is either `2` or odd -/

Lean Infoview
▼ LeanTimes.lean:4:43
No info found.
► All Messages (0)
```

Before we wrap up...



Lean FRO: Shaping the Future of Lean Development

The Lean Focused Research Organization (FRO) is a non-profit dedicated to Lean's development.

Founded in **August 2023**, the organization has 19 members.

Its mission is to enhance critical areas: **scalability, usability, documentation**, and **proof automation**.

It must reach **self-sustainability in August 2028** and become the **Lean Foundation**.

Philanthropic support is gratefully acknowledged from the **Simons Foundation**, the **Alfred P. Sloan Foundation**, **Richard Merkin**, and **Alex Gerko**.

Lean FRO: by numbers

20 releases and **4,383 pull requests** merged in the main repository only since its launch in July 2023.

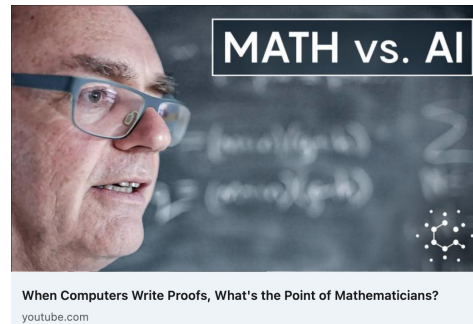
Public roadmaps: <https://lean-fro.org/about/roadmap-y2/>

Lean project was featured in multiple venues NY Times, Quanta, Scientific American, etc.



A.I. Is Coming for Mathematics, Too

For thousands of years, mathematicians have adapted to the latest advances in logic and reasoning. Are they ready for artificial intelligence?



Lean FRO: Roadmap

Lean v4.22's release will celebrate the Lean FRO's second anniversary

New Compiler - Enhanced performance and optimization

Improved do Notation - better support for reasoning about it

New Module System - Faster recompilation and better dependency management

Enhanced Proof Automation - Continue improving `bv_decide`, `grind`, `simp`

Scalability Improvements - Handle larger codebases efficiently

Literate Programming System - Seamless documentation integration

New Website - Modern interface and better resources



How can I contribute?

Help building [Mathlib](#).

Want to engage with the vibrant Lean community? Join our [Zulip channel](#).

Interested in ML kernels? Contribute to the [KLR project](#).

Want to contribute to a large formalization project? Join the [FLT formalization project](#).

Start your own open-source Lean project! Your package will be available on our registry [Reservoir](#).

Start using Lean online: [live.lean-lang.org](#)

Support the Lean FRO: Funding, partnerships, or simply advocating the project.

Conclusion

Lean is an **efficient programming language** and **proof assistant**.

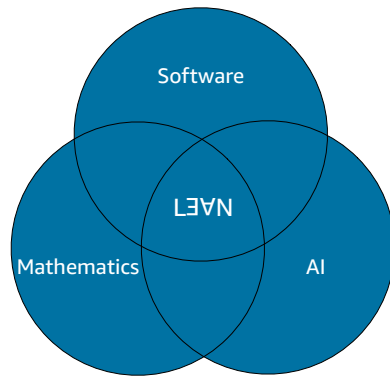
Lean is very **extensible** and is implemented in Lean.

Lean proofs are maintainable, stable, and transparent.

Progress is accelerating with the Lean FRO: module system, new compiler, new proof automation, etc.

The Mathlib community is changing how math is done.

It is not just about proving but also understanding complex objects and proofs, getting new insights, and navigating through the “thick jungles” that are **beyond our cognitive abilities**.



Thank You

<https://leanprover.zulipchat.com/>

x: @leanprover

LinkedIn: Lean FRO

Mastodon: @leanprover@functional.cafe
#leanlang, #leanprover

<https://www.lean-lang.org/>

